PROJETO.....

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO e RA'S

Vinicius de Lima Borges - 26029166

Eric Carvalho - 26029113

Gustavo Cocoloti Melo - 26028911

Douglas Silva Gadelha Macario - 26029313

Adriana Carmem - 11413

São Paulo

2025

Sumário

1 INTRODUÇÃO **3**

1.1 – Processo de desenvolvimento de Software 3

1.2 Apresentação 3

2. DESIGN SPRINT – Ideação e prototipação do desafio **3**

2.1 Desafio 3

2.2 Entender Mapear 4

2.3 Ideação – desenho da solução (trilha do usuário) 4

2.4 Prototipagem 4

3.REQUISITOS DE SISTEMA **4**

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE 4

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE 5

4. CASOS DE USO **6**

5. DIAGRAMA DE CLASSE **6**

6. ARQUITETURA DO SISTEMA **6**

6.1. Visão Geral do Sistema 6

6.2. Arquitetura Geral 6

6.3. Componentes Arquiteturais e Camadas 6

6.4. Diagrama da Arquitetura 7

6.5. Organização de Pastas e Módulos 8

6.6. Integrações Externas (se houver) 8

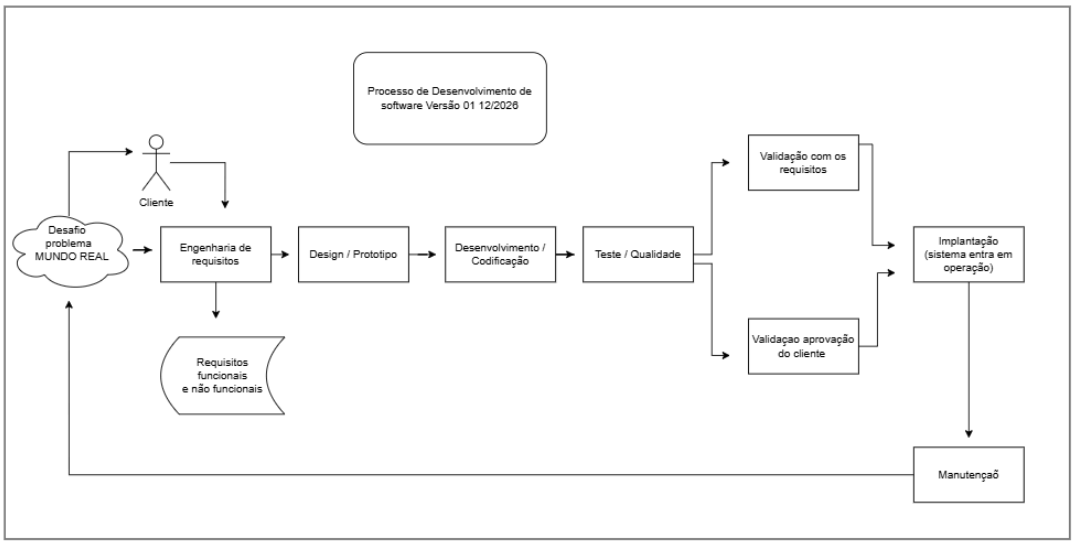
6.7. Ambiente e Implantação 9

8. REFERÊNCIAS BIBLIOGRÁFICAS **9**

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software

Exemplo (substituir) - o que foi realizado em sala no Draw.io, escolha um por equipe



1.2 Apresentação

A Messier Data & Creative é uma empresa de desenvolvimento de jogos e experiências interativas que combina tecnologia e criatividade para promover engajamento em eventos, demonstrações de produto e campanhas de marca. Como novo passo, a Messier pretende lançar uma plataforma de jogos digitais para escolas de ensino fundamental e médio. Neste Projeto Interdisciplinar, as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo. Fluxo do processo (simulação do módulo Escola): 1) Login: a escola se autentica; o aplicativo verifica se a escola possui pacote ativo e se o IP de origem (informado na simulação) está autorizado. 2) Catálogo: o aplicativo apresenta os games disponíveis no pacote adquirido pela escola. 3) Acesso: a escola simula o acesso a um game; o aplicativo registra e contabiliza o acesso no consumo do mês, respeitando o limite do pacote. Tendo.....

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

Inicial da proposta do PI - as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo.

Tempestade de Ideias:

- Banco de Dados como Base
- Planos maiores mais baratos
- Diversificação de temas
- Acesso através de CPF e senha
- Certificações
- Marketing: Trazer novos acessos
- Designer Interativo
- Teste gratuito
- Jogos offline
- Contextualização de cada curso
- Objetivos
- Linhagem de Aprendizado/Cronograma
- Rede Apoio/Suporte
- Sugestões dos usuários para melhorias
- Acessibilidade
- Gêneros
- Conexão com coordenação escolar e acompanhamento nivelado

2.2 Entender Mapear

Objetivo Geral: Desenvolver uma plataforma que contenha um catálogo de Jogos educacionais, voltado ao público do ensino fundamental e médio. A plataforma deve conter o controle de acesso dos assinantes por meio dos IP's autorizados

Como Podemos?

Como podemos estruturar o site??

Vamos estruturar a Plataforma com um Layout Interativo, de fácil entendimento.

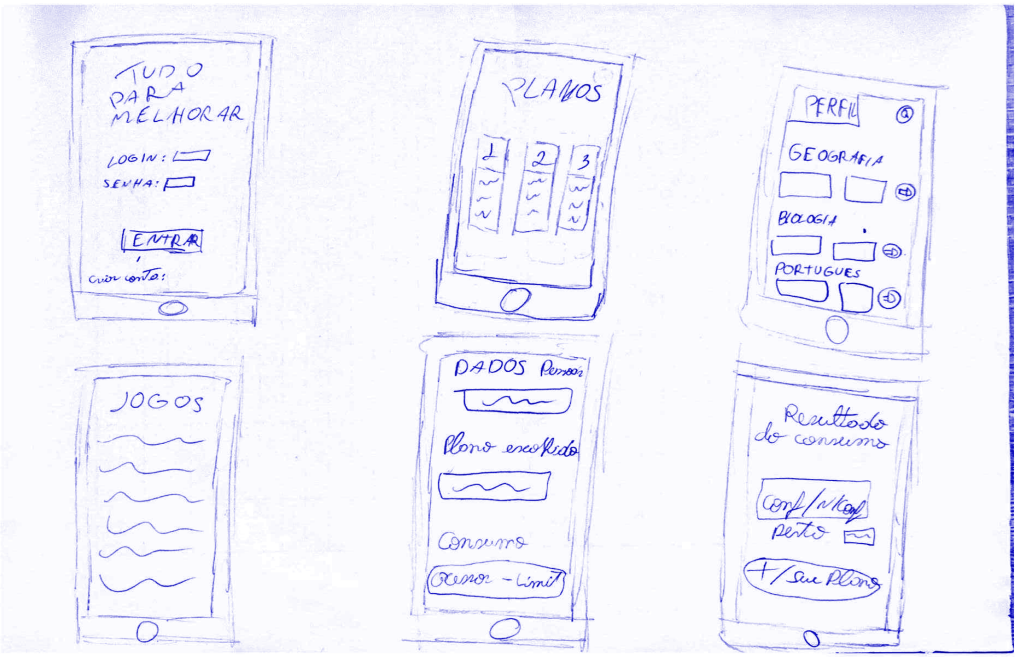
Para que o Banco de Dados e os Dados???

Para o armazenamento, controle de acessos, salvar progresso e informações pessoais

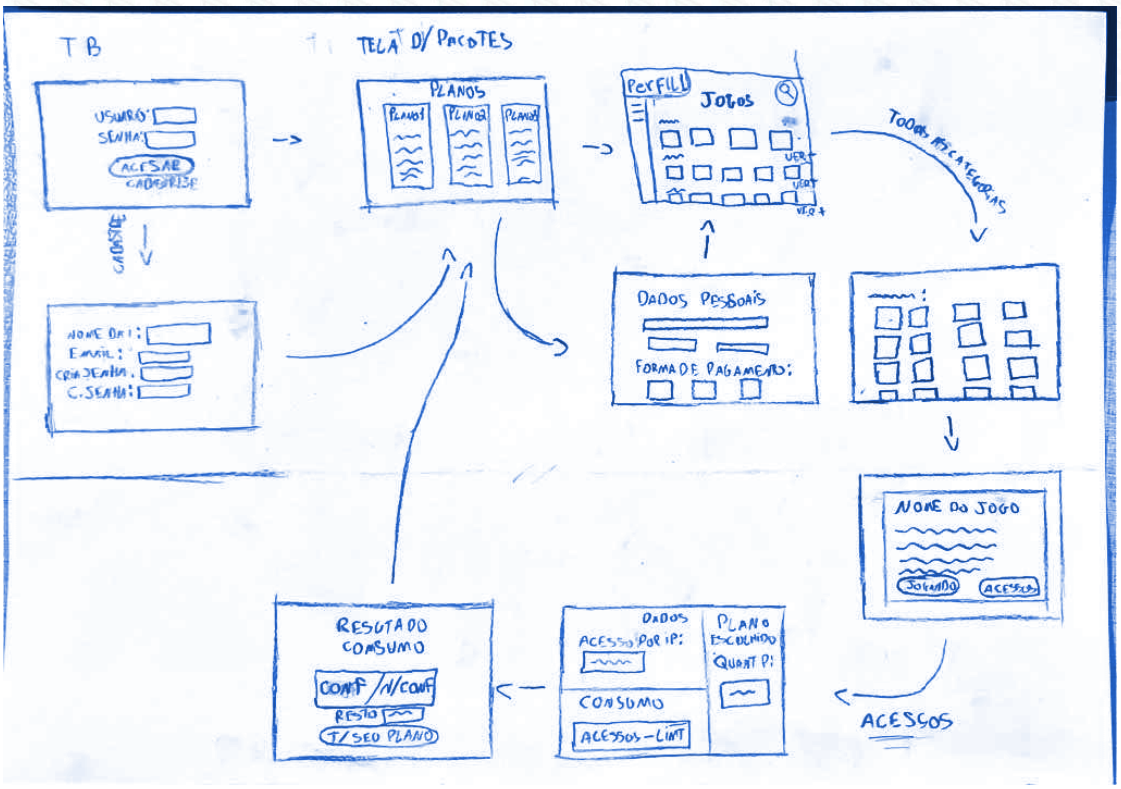
Como pode ser feito o marketing?

Através de Anúncios promocionais em redes sociais, indicações, clientes parceiros, eventos promocionais e compensar uso prolongado.

2.3 Ideação – desenho da solução (trilha do usuário)



2.4 Prototipagem



3.REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RFS01	
Função	Cadastro de Escolas
Descrição	Permite o registro de novos usuários no sistema com validação de dados
Entradas	Nome, Email, Senha, Telefone
Fonte	Formulário Web, API Externa, Banco de Dados
Saídas	Usuário cadastrado, Mensagem de sucesso
Ação	INSERT no BD + Envio de e-mail de confirmação

RFS02

Função	Processar Pagamento Cartão
Descrição	Realiza cobrança no cartão e pix
Entradas	Número Cartão, CVV, Parcela, Valor Total
Fonte	Formulário checkout
Saídas	Autorizado/Rejeitado
Ação	PDATE pedido + Email recibo

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RFS01	
Função	Nunca perde progresso
Descrição	Auto-save a cada nível
Entradas	Apenas banner discreto opcional
Fonte	
Saídas	
Ação	

RFS02	
Função	Anúncios não interrompem jogo
Descrição	
Entradas	
Fonte	
Saídas	
Ação	

4. CASOS DE USO

Apresentar 3 casos de uso do sistema

5. DIAGRAMA DE CLASSE

6. ARQUITETURA DO SISTEMA

Para a arquitetura do sistema, considere o template abaixo:

Template (esquema de arquitetura) para orientar a elaboração de um **Projeto de Arquitetura de Sistema Web**, cobrindo desde a camada de apresentação até a persistência de dados, com foco em modularidade, escalabilidade e eficiência.

6.1. Visão Geral do Sistema

- **Nome do Sistema**:
- **Objetivo do Sistema**: (Breve descrição da finalidade do sistema)
- **Público-Alvo**:
- **Principais Funcionalidades**:

6.2. Arquitetura Geral

Descrever o estilo arquitetural adotado (por exemplo, MVC, camadas, microsserviços, hexagonal, etc.):

Exemplo: Arquitetura em Camadas (Layered Architecture) com separação clara entre apresentação, lógica de negócios e persistência.

6.3. Componentes Arquiteturais e Camadas

6.3.1. Camada de Apresentação (Frontend)

- **Tecnologias**: (Ex.: HTML5, CSS3, JavaScript, React/Vue/Angular)
- **Responsabilidades**:

- Interface com o usuário
- Validação inicial de dados
- Comunicação com a API via HTTP/JSON

6.3.2. Camada de Aplicação (Backend/API)

- **Tecnologias**: (Ex.: Node.js, Django, Spring Boot, Flask, Laravel)
- **Responsabilidades**:
  - Implementação da lógica de negócio
  - Autenticação/autorização
  - Comunicação com a camada de dados
  - Exposição de endpoints RESTful/GraphQL

6.3.3. Camada de Domínio (Negócio)

- **Descrição**:
  - Contém as regras de negócio e os modelos de domínio
  - Independente das demais camadas
- **Componentes**:
  - Entidades
  - Casos de uso
  - Regras de validação

6.3.4. Camada de Persistência (Banco de Dados)

- **Tecnologias**: (Ex.: PostgreSQL, MySQL, MongoDB, SQLite)
- **Responsabilidades**:
  - Armazenamento e recuperação de dados
  - Scripts de criação e migração de banco

6.4. Diagrama da Arquitetura

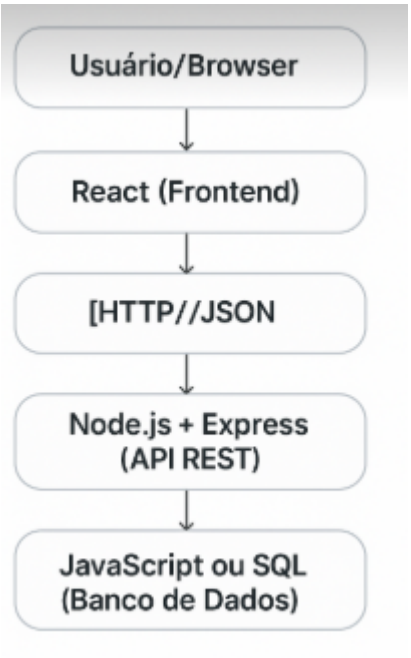
Inserir um **diagrama estrutural** (ex.: em draw.io, Lucidchart ou outro) mostrando:

- Fluxo entre frontend, backend, banco de dados
- Componentes principais
- APIs e serviços envolvidos
- Gateways, autenticação, filas de mensagens (se houver)

Exemplo textual de uma arquitetura de sistema web.

[Usuário/Browser]
|
React (Frontend)
|
[HTTP/JSON]
|
Node.js + Express (API REST)
|
JavaScript ou SQL
|
PostgreSQL (Banco de Dados)

Diagrama:



6.5. Organização de Pastas e Módulos

Descrever como o código será organizado no repositório:

```
📁 /frontend
    📁 components/
    📁 pages/
    📁 services/
📁 /backend
    📁 controllers/
    📁 services/
    📁 models/
    📁 routes/
    📁 config/
📁 /database
    📄 schema.sql
```

📄 `migrations/`

6.6. Integrações Externas (se houver)

- APIs de terceiros (pagamentos, mapas, redes sociais)
- Serviços de autenticação (OAuth2, Firebase, etc.)
- Serviços de envio (email, notificações)

6.7. Ambiente e Implantação

- **Ambientes**: desenvolvimento, homologação, produção
- **Contêineres**: Docker/Docker Compose (se usado)
- **CI/CD**: GitHub Actions, GitLab CI, Jenkins
- **Hospedagem**: Heroku, Vercel, AWS, GCP, Azure

8. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. **Engenharia de Software.** 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.